

OBJECT ORIENTED PROGRAMMING WITH C++

Unit-II: Loops, Control Statements, Structures, and Functions

Comprehensive Study Notes

HPU BCA Semester III

1. Loops in C++

1.1 Introduction to Loops

Loops are control structures that allow a block of code to be executed repeatedly based on a condition. They are essential for performing repetitive tasks efficiently.

1.2 for Loop

The **for loop** is used when the number of iterations is known in advance.

Syntax

```
for (initialization; condition; increment/decrement) {  
    // body of loop  
}
```

Example 1: Print numbers from 1 to 10

```
#include <iostream>  
using namespace std;  
  
int main()  
{  
    for (int i = 1; i <= 10; i++) {  
        cout << i << " ";  
    }  
    cout << endl;  
    return 0;  
}
```

Output:

```
1 2 3 4 5 6 7 8 9 10
```

Example 2: Print even numbers from 2 to 20

```
int main()
{
    for (int i = 2; i <= 20; i += 2) {
        cout << i << " ";
    }
    cout << endl;
    return 0;
}
```

Output:

```
2 4 6 8 10 12 14 16 18 20
```

Example 3: Sum of first N natural numbers

```
int main()
{
    int n, sum = 0;
    cout << "Enter a number: ";
    cin >> n;

    for (int i = 1; i <= n; i++) {
        sum += i;
    }
    cout << "Sum = " << sum << endl;
    return 0;
}
```

Output:

```
Enter a number: 10
Sum = 55
```

Example 4: Print multiplication table

```
int main()
{
    int n;
```

```
cout << "Enter a number: ";
cin >> n;

for (int i = 1; i <= 10; i++) {
    cout << n << " x " << i << " = " << (n * i) << endl;
}
return 0;
}
```

Output:

```
Enter a number: 5
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
```

Example 5: Nested for loop (Pattern printing)

```
int main()
{
    for (int i = 1; i <= 5; i++) {
        for (int j = 1; j <= i; j++) {
            cout << "* ";
        }
        cout << endl;
    }
    return 0;
}
```

Output:

```
*
* *
* * *
```

```
* * * *  
* * * * *
```

1.3 while Loop

The **while loop** is used when the number of iterations is not known in advance and the loop continues as long as the condition is true.

Syntax

```
while (condition) {  
    // body of loop  
}
```

Example 1: Print numbers from 1 to 10

```
int main()  
{  
    int i = 1;  
    while (i <= 10) {  
        cout << i << " ";  
        i++;  
    }  
    cout << endl;  
    return 0;  
}
```

Output:

```
1 2 3 4 5 6 7 8 9 10
```

Example 2: Factorial of a number

```
int main()  
{  
    int n, fact = 1;  
    cout << "Enter a number: ";  
    cin >> n;
```

```
int i = 1;
while (i <= n) {
    fact *= i;
    i++;
}
cout << "Factorial = " << fact << endl;
return 0;
}
```

Output:

```
Enter a number: 5
Factorial = 120
```

Example 3: Sum of digits

```
int main()
{
    int n, sum = 0;
    cout << "Enter a number: ";
    cin >> n;

    while (n > 0) {
        sum += n % 10;
        n /= 10;
    }
    cout << "Sum of digits = " << sum << endl;
    return 0;
}
```

Output:

```
Enter a number: 12345
Sum of digits = 15
```

1.4 do-while Loop

The **do-while loop** executes the body at least once before checking the condition.

Syntax

```
do {  
    // body of loop  
} while (condition);
```

Example 1: Menu-driven program

```
int main()  
{  
    int choice;  
    do {  
        cout << "--- MENU ---" << endl;  
        cout << "1. Add" << endl;  
        cout << "2. Subtract" << endl;  
        cout << "3. Multiply" << endl;  
        cout << "4. Exit" << endl;  
        cout << "Enter your choice: ";  
        cin >> choice;  
  
        switch (choice) {  
            case 1: cout << "Addition selected" << endl; break;  
            case 2: cout << "Subtraction selected" << endl; break;  
            case 3: cout << "Multiplication selected" << endl; break;  
            case 4: cout << "Exiting..." << endl; break;  
            default: cout << "Invalid choice" << endl;  
        }  
    } while (choice != 4);  
    return 0;  
}
```

Example 2: Print numbers from 1 to 5

```
int main()  
{  
    int i = 1;  
    do {  
        cout << i << " ";  
        i++;  
    } while (i <= 5);  
    cout << endl;  
    return 0;  
}
```

Output:

```
1 2 3 4 5
```

1.5 Loop Comparison

Loop	When to Use	Executes at Least Once
for	Known number of iterations	No
while	Unknown number of iterations	No
do-while	Need to execute at least once	Yes

2. Control Statements

2.1 if Statement

The **if statement** executes a block of code if the condition is true.

Syntax

```
if (condition) {  
    // code to execute if condition is true  
}
```

Example: Check if number is positive

```
int main()  
{  
    int num;  
    cout << "Enter a number: ";  
    cin >> num;  
  
    if (num > 0) {  
        cout << num << " is positive." << endl;  
    }  
    return 0;  
}
```

Output:

```
Enter a number: 5  
5 is positive.
```

2.2 if-else Statement

The **if-else statement** executes one block if the condition is true and another if it is false.

Syntax

```
if (condition) {  
    // code if true  
} else {  
    // code if false  
}
```

Example 1: Check even or odd

```
int main()  
{  
    int num;  
    cout << "Enter a number: ";  
    cin >> num;  
  
    if (num % 2 == 0) {  
        cout << num << " is even." << endl;  
    } else {  
        cout << num << " is odd." << endl;  
    }  
    return 0;  
}
```

Output:

```
Enter a number: 7  
7 is odd.
```

Example 2: Check positive or negative

```
int main()  
{  
    int num;  
    cout << "Enter a number: ";  
    cin >> num;  
  
    if (num >= 0) {  
        cout << "Positive or Zero" << endl;  
    } else {  
        cout << "Negative" << endl;  
    }  
}
```

```
    return 0;
}
```

2.3 else-if Ladder

The **else-if ladder** is used to test multiple conditions.

Syntax

```
if (condition1) {
    // code block 1
} else if (condition2) {
    // code block 2
} else if (condition3) {
    // code block 3
} else {
    // default code block
}
```

Example: Grade Calculation

```
int main()
{
    int marks;
    cout << "Enter your marks: ";
    cin >> marks;

    if (marks >= 90) {
        cout << "Grade: A+ (Excellent)" << endl;
    } else if (marks >= 75) {
        cout << "Grade: A (Very Good)" << endl;
    } else if (marks >= 60) {
        cout << "Grade: B (Good)" << endl;
    } else if (marks >= 45) {
        cout << "Grade: C (Average)" << endl;
    } else {
        cout << "Grade: F (Fail)" << endl;
    }
    return 0;
}
```

Output:

```
Enter your marks: 85
Grade: A (Very Good)
```

2.4 Nested if

Nested if is an if statement inside another if statement.

Example: Find largest of three numbers

```
int main()
{
    int a, b, c;
    cout << "Enter three numbers: ";
    cin >> a >> b >> c;

    if (a >= b) {
        if (a >= c) {
            cout << "Largest: " << a << endl;
        } else {
            cout << "Largest: " << c << endl;
        }
    } else {
        if (b >= c) {
            cout << "Largest: " << b << endl;
        } else {
            cout << "Largest: " << c << endl;
        }
    }
    return 0;
}
```

2.5 switch Statement

The **switch statement** is used for multi-way branching based on the value of an expression.

Syntax

```
switch (expression) {  
    case value1:  
        // code block 1  
        break;  
    case value2:  
        // code block 2  
        break;  
    // more cases...  
    default:  
        // default code block  
}
```

Example: Simple Calculator

```
int main()  
{  
    char op;  
    double a, b;  
  
    cout << "Enter operator (+, -, *, /): ";  
    cin >> op;  
    cout << "Enter two numbers: ";  
    cin >> a >> b;  
  
    switch (op) {  
        case '+':  
            cout << a << " + " << b << " = " << (a + b) << endl;  
            break;  
        case '-':  
            cout << a << " - " << b << " = " << (a - b) << endl;  
            break;  
        case '*':  
            cout << a << " * " << b << " = " << (a * b) << endl;  
            break;  
        case '/':  
            if (b != 0)  
                cout << a << " / " << b << " = " << (a / b) << endl;  
            else  
                cout << "Division by zero!" << endl;  
            break;  
        default:  
            cout << "Invalid operator!" << endl;  
    }  
}
```

```
    return 0;  
}
```

Output:

```
Enter operator (+, -, *, /): *  
Enter two numbers: 10 5  
10 * 5 = 50
```

2.6 Conditional Operator (Ternary Operator)

The **conditional operator** is a shorthand for if-else.

Syntax

```
condition ? expression1 : expression2;
```

Example

```
int main()  
{  
    int a = 10, b = 20;  
    int max = (a > b) ? a : b;  
    cout << "Maximum: " << max << endl;  
  
    int num = 5;  
    cout << num << " is "  
        << ((num % 2 == 0) ? "Even" : "Odd") << endl;  
    return 0;  
}
```

Output:

```
Maximum: 20  
5 is Odd
```

3. Jump Statements: break, continue, goto

3.1 break Statement

The **break statement** is used to exit from a loop or switch statement.

Example

```
int main()
{
    // break in loop
    for (int i = 1; i <= 10; i++) {
        if (i == 5) {
            cout << "Breaking at " << i << endl;
            break;
        }
        cout << i << " ";
    }
    cout << "Loop ended" << endl;

    // break in switch
    int choice = 2;
    switch (choice) {
        case 1: cout << "Case 1"; break;
        case 2: cout << "Case 2"; break;
        case 3: cout << "Case 3"; break;
    }
    return 0;
}
```

Output:

```
1 2 3 4 Breaking at 5
Loop ended
Case 2
```

3.2 continue Statement

The **continue statement** skips the current iteration and continues with the next iteration.

Example

```
int main()
{
    // Print odd numbers from 1 to 10
    for (int i = 1; i <= 10; i++) {
        if (i % 2 == 0) {
            continue;
        }
        cout << i << " ";
    }
    cout << endl;

    // Skip 5 and 7
    for (int i = 1; i <= 10; i++) {
        if (i == 5 || i == 7) {
            continue;
        }
        cout << i << " ";
    }
    return 0;
}
```

Output:

```
1 3 5 7 9
1 2 3 4 6 8 9 10
```

3.3 goto Statement

The **goto statement** transfers control to a labeled statement.

Example

```
int main()
{
    int num;
```



```
    cout << "Enter a number: ";  
    cin >> num;  
  
    if (num < 0) {  
        goto negative;  
    }  
    cout << "Number is positive: " << num << endl;  
    goto end;  
  
negative:  
    cout << "Number is negative: " << num << endl;  
  
end:  
    cout << "Program ended." << endl;  
    return 0;  
}
```

Output 1:

```
Enter a number: 10  
Number is positive: 10  
Program ended.
```

Output 2:

```
Enter a number: -5  
Number is negative: -5  
Program ended.
```

Note: The `goto` statement is generally avoided as it makes code less readable.

4. Structures

4.1 What is a Structure?

A **structure** is a user-defined data type that groups together different data types into a single unit.

Syntax

```
struct structure_name {  
    data_type member1;  
    data_type member2;  
    // more members...  
};
```

Example: Defining a Student structure

```
struct Student {  
    int rollNo;  
    string name;  
    float marks;  
};
```

4.2 Accessing Structure Members

Structure members are accessed using the **dot operator** (.).

```
#include <iostream>  
#include <string>  
using namespace std;  
  
struct Student {  
    int rollNo;  
    string name;  
    float marks;  
};
```

```
int main()
{
    Student s1; // Creating structure variable

    s1.rollNo = 101;
    s1.name = "Rahul";
    s1.marks = 85.5;

    cout << "Roll No: " << s1.rollNo << endl;
    cout << "Name: " << s1.name << endl;
    cout << "Marks: " << s1.marks << endl;
    return 0;
}
```

Output:

```
Roll No: 101
Name: Rahul
Marks: 85.5
```

4.3 Array of Structures

```
int main()
{
    Student students[3]; // Array of 3 Student structures

    // Input data
    for (int i = 0; i < 3; i++) {
        cout << "Enter details for student " << i+1 << endl;
        cout << "Roll No: ";    cin >> students[i].rollNo;
        cout << "Name: ";      cin >> students[i].name;
        cout << "Marks: ";     cin >> students[i].marks;
    }

    // Display data
    cout << "nStudent Details:" << endl;
    for (int i = 0; i < 3; i++) {
        cout << "Roll: " << students[i].rollNo
            << ", Name: " << students[i].name
            << ", Marks: " << students[i].marks << endl;
    }
}
```

```
    return 0;  
}
```

4.4 Structure within a Structure (Nested Structure)

```
struct Address {  
    string city;  
    string state;  
    int pincode;  
};  
  
struct Employee {  
    int empId;  
    string name;  
    double salary;  
    Address addr; // Nested structure  
};  
  
int main()  
{  
    Employee e1;  
    e1.empId = 101;  
    e1.name = "Priya";  
    e1.salary = 50000;  
    e1.addr.city = "Shimla";  
    e1.addr.state = "Himachal Pradesh";  
    e1.addr.pincode = 171001;  
  
    cout << "Employee ID: " << e1.empId << endl;  
    cout << "Employee Name: " << e1.name << endl;  
    cout << "City: " << e1.addr.city << endl;  
    cout << "State: " << e1.addr.state << endl;  
    cout << "Pincode: " << e1.addr.pincode << endl;  
    return 0;  
}
```

Output:

```
Employee ID: 101  
Employee Name: Priya  
City: Shimla
```

State: Himachal Pradesh
Pincode: 171001

4.5 Enumerated Data Type (enum)

enum is used to define a set of named integer constants.

Syntax

```
enum enum_name { value1, value2, value3, ... };
```

Example

```
enum Days { Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday };
enum Colors { Red = 1, Green = 2, Blue = 3 };
enum Months { Jan = 1, Feb, Mar, Apr, May, Jun,
              Jul, Aug, Sep, Oct, Nov, Dec };

int main()
{
    Days today = Tuesday;
    Colors fav = Green;
    Months month = Mar;

    cout << "Today: " << today << endl;           // Output: 2
    cout << "Favorite Color: " << fav << endl;       // Output: 2
    cout << "Month: " << month << endl;             // Output: 3

    if (today < Friday)
        cout << "Midweek" << endl;

    return 0;
}
```

Output:

```
Today: 2
Favorite Color: 2
Month: 3
Midweek
```

5. Functions in C++

5.1 Simple Functions

A **function** is a block of code that performs a specific task.

Syntax

```
return_type function_name(parameters) {  
    // body of function  
    return value; // optional  
}
```

Example

```
// Function to display a greeting  
void greet() {  
    cout << "Hello! Welcome to C++" << endl;  
}  
  
// Function to add two numbers  
int add(int a, int b) {  
    return a + b;  
}  
  
int main()  
{  
    greet();  
    int sum = add(10, 20);  
    cout << "Sum = " << sum << endl;  
    return 0;  
}
```

Output:

```
Hello! Welcome to C++  
Sum = 30
```

5.2 Passing Arguments to Functions

Pass by Value

A copy of the argument is passed; changes inside the function do not affect the original.

Pass by Reference

A reference to the argument is passed; changes inside the function affect the original.

```
// Pass by value
void square(int n) {
    n = n * n;
    cout << "Square: " << n << endl;
}

// Pass by reference
void doubleValue(int &n) {
    n = n * 2;
}

int main()
{
    int a = 5;
    square(a);
    cout << "a: " << a << endl; // a is still 5 (pass by value)

    int b = 10;
    doubleValue(b);
    cout << "b: " << b << endl; // b becomes 20 (pass by reference)
    return 0;
}
```

Output:

```
Square: 25
a: 5
b: 20
```

5.3 Returning Values from Functions

```
int add(int a, int b) {  
    return a + b;  
}  
  
int subtract(int a, int b) {  
    return a - b;  
}  
  
int max(int a, int b) {  
    if (a > b) return a;  
    else return b;  
}  
  
int main()  
{  
    cout << "Add: " << add(10, 5) << endl;  
    cout << "Subtract: " << subtract(10, 5) << endl;  
    cout << "Max: " << max(10, 5) << endl;  
    return 0;  
}
```

Output:

```
Add: 15  
Subtract: 5  
Max: 10
```

5.4 Reference Arguments (Swapping using References)

```
void swap(int &a, int &b) {  
    int temp = a;  
    a = b;  
    b = temp;  
}  
  
int main()  
{  
    int x = 10, y = 20;  
    cout << "Before swap: x = " << x << ", y = " << y << endl;  
    swap(x, y);  
    cout << "After swap: x = " << x << ", y = " << y << endl;  
}
```



```
    return 0;  
}
```

Output:

```
Before swap: x = 10, y = 20  
After swap: x = 20, y = 10
```

5.5 Function Overloading

Function overloading allows multiple functions with the same name but different parameters.

```
int add(int a, int b) {  
    return a + b;  
}  
  
double add(double a, double b) {  
    return a + b;  
}  
  
int add(int a, int b, int c) {  
    return a + b + c;  
}  
  
int main()  
{  
    cout << add(10, 5) << endl;           // Calls int add(int, int)  
    cout << add(2.5, 3.35) << endl;       // Calls double add(double, double)  
    cout << add(1, 2, 3) << endl;         // Calls int add(int, int, int)  
    return 0;  
}
```

Output:

```
15  
5.85  
6
```

5.6 Default Arguments

```
void display(string name, int age = 18) // Default value for age
{
    cout << "Name: " << name << ", Age: " << age << endl;
}

int main()
{
    display("Rahul"); // Uses default age (18)
    display("Priya", 22); // Uses provided age (22)
    return 0;
}
```

Output:

```
Name: Rahul, Age: 18
Name: Priya, Age: 22
```

5.7 Inline Functions

Inline functions suggest the compiler to expand the function code at the call site, eliminating function call overhead for small functions.

```
inline int square(int x) {
    return x * x;
}

int main()
{
    cout << "Square of 5: " << square(5) << endl;
    cout << "Square of 10: " << square(10) << endl;
    return 0;
}
```

Output:

```
Square of 5: 25
Square of 10: 100
```

5.8 Storage Classes

Storage Class	Keyword	Scope	Lifetime	Default Value
Auto	auto	Local	Function	Garbage
Register	register	Local	Function	Garbage
Static	static	Local/Global	Program	0
Extern	extern	Global	Program	0

Static Variable Example

```
void countCalls() {
    static int count = 0; // Initialized only once
    count++;
    cout << "Function called " << count << " times" << endl;
}

int main()
{
    countCalls(); // Output: Function called 1 times
    countCalls(); // Output: Function called 2 times
    countCalls(); // Output: Function called 3 times
    return 0;
}
```

5.9 Scope Resolution Operator (::)

The **scope resolution operator** is used to access global variables when they are hidden by local variables.

```
int x = 100; // Global variable

int main()
{
    int x = 50; // Local variable

    cout << "Local x: " << x << endl; // Accesses local x
}
```

```
    cout << "Global x: " << ::x << endl;    // Accesses global x using ::  
    return 0;  
}
```

Output:

```
Local x: 50  
Global x: 100
```

6. Arrays in C++

6.1 One-Dimensional Arrays

An **array** is a collection of elements of the same data type stored in contiguous memory locations.

```
int main()
{
    // Declaration and initialization
    int arr[5] = {10, 20, 30, 40, 50};

    // Accessing elements
    for (int i = 0; i < 5; i++) {
        cout << "arr[" << i << "] = " << arr[i] << endl;
    }
    return 0;
}
```

6.2 Two-Dimensional Arrays

```
int main()
{
    // Declaration and initialization
    int matrix[2][3] = {
        {1, 2, 3},
        {4, 5, 6}
    };

    // Accessing elements
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 3; j++) {
            cout << matrix[i][j] << " ";
        }
        cout << endl;
    }
    return 0;
}
```

6.3 Passing Arrays to Functions

```
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

int main()
{
    int numbers[] = {10, 20, 30, 40, 50};
    int size = sizeof(numbers) / sizeof(numbers[0]);
    printArray(numbers, size);
    return 0;
}
```

6.4 C++ String Class

```
#include <iostream>
#include <string>
using namespace std;

int main()
{
    string s1 = "Hello";
    string s2 = " World";
    string s3 = s1 + s2;           // Concatenation

    cout << "s1: " << s1 << endl;
    cout << "s3: " << s3 << endl;
    cout << "Length: " << s3.length() << endl;
    return 0;
}
```

7. Questions for Practice

Short Answer Questions (2-5 Marks)

1. What is a loop? Name the types of loops in C++.
2. Explain the difference between while and do-while loops.
3. What is the switch statement? Give its syntax.
4. What is the difference between break and continue?
5. What is a structure in C++?
6. What is an enumerated data type?
7. What is the difference between pass by value and pass by reference?
8. What is function overloading?
9. What is the scope resolution operator?
10. What are storage classes in C++?

Long Answer Questions (10 Marks)

1. Explain loops in C++ with syntax and examples.
2. Explain decision control statements with examples.
3. Explain structures and nested structures with examples.
4. Explain functions in C++ with different parameter passing methods.
5. Explain the switch statement with a suitable example.

Program Writing Questions (5 Marks)

1. Write a program to print Fibonacci series up to n terms.
2. Write a program to check if a number is prime or not.
3. Write a program to accept student details using structure.
4. Write a program using functions to calculate area of rectangle, circle, and triangle.
5. Write a program to display multiplication table using for loop.

Objective Questions (1 Mark)

1. The loop that executes at least once is:
(a) for (b) while (c) do-while (d) None
2. The `break` statement:
(a) Exits loop (b) Continues loop (c) Both (d) None
3. The `continue` statement:
(a) Exits loop (b) Continues loop (c) Both (d) None
4. Structures in C++ are:
(a) User-defined data types (b) Built-in data types (c) Both (d) None
5. Function overloading means:
(a) Same name, different parameters (b) Different name, same parameters (c) Both
(d) None

Quick Revision

Loops Summary

Loop	Syntax	Use Case
for	for(init; cond; inc/dec) { }	Known iterations
while	while(cond) { }	Unknown iterations
do-while	do { } while(cond);	At least once execution

Control Statements

Statement	Purpose
if	Execute code based on condition
if-else	Two-way decision making
else-if ladder	Multi-way decision making
switch	Multi-way branching by value
ternary (?:)	Shorthand for if-else

Functions Summary

Feature	Description
Pass by value	Function works with a copy; original unchanged
Pass by reference	Function works with original; changes reflect
Inline functions	Expanded at call site to reduce overhead

Default arguments	Parameters with default values
Function overloading	Same name, different parameter lists

End of Unit-II Notes